

SignVLM for Urdu/Pakistani Sign Language — Architecture & Pipeline Summary

Reference paper: Luqman (2025), *SignVLM: A pre-trained large vision model for sign language recognition*, PeerJ Comput. Sci. 11:e3112 (PMC12453763).

Our run: PSL-104 signer-disjoint fine-tune — full metrics in `docs/retrain_details/signvml_report/SignVLM_Retrain_Report.md`.

Training details below come from the actual training notebook (`notebooks/SignVLM_Colab_Training.ipynb`) and the dataloader code (`video_dataset/`), not from the paper.

1. Architecture

SignVLM = **frozen CLIP image encoder + lightweight EVL-style temporal decoder + linear classifier**. Only the decoder and classifier are trained; the visual backbone is never updated.

Stage	What we used (Cell 5, EVLTransformer)	Notes
Visual backbone	OpenAI CLIP ViT-L/14 (ViT-L/14-lnpre , weights from ViT-L-14.pt), mode <code>freeze_fp16</code>	Frozen, run in fp16; encodes each frame independently
Temporal decoder	4 decoder layers, qkv_dim 1024, 16 heads, <code>decoder_mlp_dropout=0.5</code>	EVL-style: per-layer depthwise temporal conv + temporal positional embeddings + cross-frame attention
Head	LayerNorm → Dropout (<code>cls_dropout=0.5</code>) → Linear → 104 logits	PSL-104 (English gloss words + Urdu alphabet letters)

Input tensor: `[B, 3, T=24, 224, 224]` — frames are flattened to 2D images for the backbone, the last 4 layers' token features are reshaped per-frame and fed to the decoder; prediction comes from the decoder's final class token. The paper reports **58.6M trainable parameters** (decoder + head only).

2. Which pretrained model? (the “Arabic model” question)

No — there is no Arabic (or any sign-language) pretrained SignVLM checkpoint, and we did not start from one. The “pre-trained large vision model” in the name is **generic CLIP**, not a model pretrained on sign language. In the paper, each language (Arabic KArSL, American WLASL-100, Turkish AUTSL, Argentinian LSA64) is fine-tuned **independently, always starting from the same frozen CLIP encoder** with a freshly initialized decoder — there is no cross-language transfer and no released per-language checkpoint.

We did exactly the same: **OpenAI CLIP ViT-L/14 frozen backbone + decoder/head trained from random initialization on our PSL-104 data**. No KArSL/Arabic weights were involved.

3. Preprocessing (offline, dataset construction)

- Clip trimming** — `augmentation_and_preprocessing/preprocessing/cropping.py` : ffmpeg-based in-place trimming of recording lead-in/lead-out on over-long clips (~0.5 s from clip edges).
- Signer ROI cropping** — `mediapipe_roi.py` + `batch_roi_extraction.py` : MediaPipe person/upper-body detection per frame, padded bounding box with EMA smoothing across frames, letterbox resize to a square crop centred on the signer. (Per-class clip preparation was carried further by the second group on top of this.)
- Per-class folders + splits** — `<split>/<label>/<clip>.mp4`; Cell 4 emits `train.tsv / val.tsv / test.tsv` (path + integer label). Split sizes: **4,368 train / 1,244 val / 1,248 test**, test set **signer-disjoint** (a held-out signer).

4. **Frame extraction** — ffmpeg extracts JPEG frames per clip (idempotent Cell 4 step), so the dataloader reads frames directly (`frames_available=1`) instead of decoding video.

4. How variable-length / variable-size clips become fixed model input

The model itself **always receives exactly** `[3, 24, 224, 224]` — all variability is absorbed by the dataloader (`video_dataset/dataset.py`):

Temporal (different clip lengths, dropped frames). Training uses `_random_sample_frame_idx` (`dataset.py:453`): if the clip has $\geq 4 \times (24-1) + 1 = 93$ frames, a random start is chosen and 24 frames are taken at stride 4; if the clip is **shorter** (e.g. after the offline frame-drop augmentation, or a short recording), `frames_downUpSamples` (`dataset.py:475`) linearly re-indexes the available N frames onto 24 target slots (`index[i] = int(N/24 * i)`) — repeating frames when $N < 24$ and skipping when $24 \leq N < 93$. So a clip that lost 5–15% of its frames to augmentation produces the **same tensor shape**; the only effect is that the surviving content is sampled at slightly different time points — i.e. the frame-drop augmentation acts as a **signing-speed / timing perturbation**, teaching the temporal decoder to tolerate irregular frame timing. At evaluation, `_generate_temporal_crops` (`dataset.py:411`) instead pads short clips by repeating the last frame and takes 3 evenly spaced temporal windows (`num_temporal_views=3`), averaging the softmax over views.

Spatial (different resolutions). Training: `random_resized_crop(frames, 224, 224)` (`transform.py:544`) samples a random sub-region covering 8–100% of the source frame area with aspect ratio between 3:4 and 4:3, then bilinearly resizes it to 224×224 — any input resolution works. Evaluation: an aspect-preserving bilinear resize sets the short side to 224 (`dataset.py:308–317`), followed by a center crop (`num_spatial_views=1`). 224×224 is required because CLIP ViT-L/14 cuts the image into fixed 14×14 -pixel patches ($16 \times 16 = 256$ tokens + CLS) matching its pretrained positional embeddings.

Normalization (both paths): CLIP statistics passed explicitly in Cell 4 — `mean=[0.48145466, 0.4578275, 0.40821073]`, `std=[0.26862954, 0.26130258, 0.27577711]`.

5. Augmentation

(a) Offline video-level augmentation — `augmentation_combined.py` generates extra `.mp4` files per class folder; each augmented clip applies **1 or 2 randomly chosen techniques** from:

Technique	Range
Lower-body crop	remove bottom 20% of frame
Scale	0.8× (down) or 1.2× (up)
Brightness	×U(0.7, 1.4)
Rotation	±10°
Brightness/saturation/hue shift	preset (90, 110, +10) or (85, 115, −10)
Temporal frame jitter	offset 1–3 frames
Random frame drop	5–15% of frames

Output clips are padded to a minimum of 30 frames. The Colab batch notebook (`SignVLM_Data_Augmentation_Colab.ipynb`) adds **9 augmented clips per class** (`*_aug_16 ... _aug_24`) to the validation/test/unseen splits; the train split's `*_aug_1 ... _aug_15` clips came from an earlier pass.

(b) Online training-time augmentation (what the code actually applies, train split only):

- **Random temporal window** — random 24-frame window at stride 4 per epoch (see §4).
- **Random resized crop** — random 8–100%-area crop, aspect 3:4–4:3, resized to 224×224.
- **RandAugment (`auto_augment`)** — **removed for this training run** (set to `None`).

Evaluation applies none of the above — deterministic resize + center crop, 3 temporal views averaged.

6. Training settings (as configured in the notebook)

Setting	Value	Where
Backbone mode	<code>freeze_fp16</code> (frozen CLIP, fp16)	Cell 5
Epochs	<code>46 (NUM_EPOCHS = round(10,000 legacy steps / 218 steps-per-epoch))</code>	Cell 4
Steps	<code>218 steps/epoch × 46 = 10,028 total (drop_last=True)</code>	Cell 4
Batch size	<code>20 (max_speed profile), batch_split=1</code> on A100 (2 on smaller GPUs)	Cells 4/6
Optimizer	<code>AdamW, lr = 4e-5, weight_decay = 0.05</code>	Cell 6
LR schedule	<code>CosineAnnealingLR(T_max = 10,028)</code> → decays to ~1e-8, no warmup	Cell 6
Loss	<code>CrossEntropyLoss</code>	Cell 7
Precision	<code>AMP fp16 (GradScaler + autocast)</code>	Cells 6/7
Dropout	<code>cls_dropout=0.5, decoder_mlp_dropout=0.5</code>	Cell 5
Loader	<code>num_workers=6, prefetch_factor=4</code> , persistent workers, local <code>/content</code> data cache	Cell 4
Validation	every 5th epoch (<code>EVAL_EVERY_N_EPOCHS=5</code>), eval-mode, 1 spatial × 3 temporal views, batch 1	Cells 6/7
Checkpointing	every epoch (<code>SAVE_EVERY_N_EPOCHS=1</code>), auto-resume from latest on Drive	Cells 6/7

Results (best checkpoint, epoch 45): train 98.01% · validation **94.53%** · same-signer test 93.75% · **signer-disjoint test 78.12%** top-1 (val top-5 ≈ 99.4%). The −15.6-point signer-shift gap concentrates in a few visually adjacent sign cliques (Hear/Healthy/He_or_she; Why/Where/Four) — see the retrain report §5–6.